



Git Workflow — Distribuciones Enzo

Estructura de ramas

main ← producción (lo que está en vivo)

└─ dev ← desarrollo activo (acá trabajás día a día)

└─ feature/nombre ← funcionalidades nuevas

└─ fix/nombre ← corrección de bugs

└─ hotfix/nombre ← arreglo urgente directo a main

Setup inicial (una sola vez)

git init

git add .

git commit -m "chore: setup inicial del proyecto"

git branch -M main

git remote add origin https://github.com/TU_USUARIO/distribuciones-enzo.git

git push -u origin main

Crear la rama dev

git checkout -b dev

git push -u origin dev

Flujo diario de trabajo

1. Siempre arrancá desde dev actualizado

```
git checkout dev
```

```
git pull origin dev
```

2. Crear una rama para el trabajo nuevo

Para una funcionalidad nueva

```
git checkout -b feature/variantes-2d
```

Para un bug fix

```
git checkout -b fix/precio-csv-parsing
```

3. Trabajar, guardar cambios con commits pequeños

```
git add admin.js
```

```
git commit -m "fix: corregir parseo de precios en formato US del CSV"
```

```
git add script.js
```

```
git commit -m "fix: proteger JSON.parse del carrito con try/catch"
```

4. Cuando terminás, mergear a dev

```
git checkout dev
```

```
git merge fix/precio-csv-parsing
```

```
git push origin dev
```

```
git branch -d fix/precio-csv-parsing # limpiar la rama
```

5. Cuando dev está listo para producción → mergear a main

```
git checkout main
```

```
git merge dev
```

```
git tag v1.2.0 -m "Release: bugfixes precios CSV y memoria admin"
```

```
git push origin main --tags
```

Formato de commit messages (Conventional Commits)

tipo(scope): descripción corta en minúscula

[cuerpo opcional — qué y por qué, no el cómo]

[footer opcional — referencias a issues]

Tipos:

Tipo	Cuándo usarlo
feat	Nueva funcionalidad
fix	Corrección de bug
refactor	Cambio de código sin fix ni feature nuevo
style	Cambios visuales/CSS sin lógica
chore	Tareas de mantenimiento (deps, config)
docs	Solo documentación
perf	Mejoras de rendimiento
hotfix	Arreglo urgente directo a producción

Ejemplos reales de este proyecto:

```
git commit -m "fix(csv): corregir parseo de precios formato US (3,000.00 → 3000)"
```

```
git commit -m "fix(admin): memory leak en listeners onSnapshot de Firestore"
```

```
git commit -m "fix(admin): categoría ignorada cuando buscador está vacío"
```

```
git commit -m "fix(admin): formulario nuevo no limpiaba variantes del anterior"
```

```
git commit -m "feat(variantes): selector 2D con modal en catálogo"
```

```
git commit -m "perf(cache): reducir TTL de caché a 5 minutos"
```

```
git commit -m "style(admin): mejorar editor de variantes con tabla visual"
```

Comandos útiles del día a día

Ver en qué rama estás y qué cambió

```
git status
```

```
git diff admin.js
```

Ver historial de commits limpio

```
git log --oneline --graph --all
```

Deshacer el último commit (pero mantener los cambios)

```
git reset --soft HEAD~1
```

Guardar trabajo temporalmente sin commitear

```
git stash
```

```
git stash pop
```

Ver qué cambió entre dos commits

```
git diff v1.1.0 v1.2.0 -- admin.js
```

Commit de hoy — bugfixes (copiar y pegar)

```
git checkout dev
```

```
git add admin.js script.js
```

```
git commit -m "fix: corregir 7 bugs críticos en admin y catálogo"
```

- fix(admin): 4 listeners onSnapshot acumulaban memoria en cada cambio de pestaña
- fix(admin): filtro de categoría ignorado cuando el buscador estaba vacío
- fix(admin): formulario nuevo mostraba variantes del producto editado anterior
- fix(admin): sort de tabla se perdía al actualizar stock en tiempo real
- fix(admin): botón eliminar roto si nombre del producto tenía apóstrofe
- fix(carrito): JSON.parse sin try/catch crasheaba si localStorage estaba corrupto
- fix(pedidos): items en Firestore incluían campos internos innecesarios"

git push origin dev